Microservizi con SpringBoot

I microservizi sono servizi che rappresentano un determinato dominio del business e che possono essere rilasciati in modo indipendente. Le funzionalità di un microservizio sono rese accessibili agli altri servizi tramite la rete, fornendo un endpoint (come ad esempio una coda o una chiamata REST, *richiesta HTTP per ottenere o modificare dati*, basata su protocolli di comunicazione). I microservizi si basano sul concetto di information hiding, ovvero nascondono al loro interno il maggior numero di informazioni possibile, esponendo solo lo stretto necessario attraverso le interfacce. Spesso per una singola funzionalità esistono più tipi di interfacce, e per rappresentare graficamente un microservizio si usa tipicamente il simbolo di un esagono.

Concetti chiave dei microservizi

→ **Deploy indipendente**: il rilascio di un microservizio avviene in totale indipendenza rispetto agli altri.

→ **Accoppiamento lasco e possesso dello stato**: per permettere il deploy indipendente, i microservizi non condividono il database, ma possiedono un proprio stato. Se un microservizio ha bisogno di accedere ai dati di un altro, deve fargliene richiesta apposita tramite la sua interfaccia.

→ **Modellazione del business**: ogni microservizio modella un dominio del business specifico (un'attività o un lavoro).

→ **Dimensione**: non devono essere troppo grandi. La prassi è iniziare con uno o pochi microservizi, per poi aggiungerne di nuovi al fine di rispettare l'information hiding e mantenere bassa la complessità.

Vantaggi dei microservizi

L'adozione di questa architettura porta diversi benefici:

- permettono l'uso di un'eterogeneità di tecnologie differenti.
- robustezza.
- scalabilità.
- facilità di deploy.
- componibilità.

Stili di comunicazione fra microservizi

I microservizi possono comunicare tra loro adottando diversi stili:

- **bloccante**: un microservizio chiama un altro microservizio e ferma la sua esecuzione per aspettare la risposta.
- **asincrono**: un microservizio effettua una chiamata a un altro, ma non si blocca e continua a portare avanti altre operazioni.
- **richiesta-risposta**: un microservizio invia una specifica richiesta a un altro e attende di ricevere una risposta contenente l'esito.
- **guidato da eventi** (event-driven): i microservizi emettono degli eventi che altri microservizi consumano per poter reagire di conseguenza. Il microservizio che genera l'evento non conosce l'identità di chi lo consumerà.

- **dati condivisi**: i microservizi collaborano leggendo e scrivendo su una sorgente di dati condivisa.

Scelte tecnologiche

Per implementare l'architettura a microservizi, esistono diverse tecnologie e framework di comunicazione:

- **Framework per chiamate remote**: permettono la comunicazione tra processi remoti, come ad esempio SOAP e gRPC.
  - **SOAP** = *Simple Object Access Protocol*.
  - **gRPC** = *gRPC Remote Procedure Calls*.
- **REST**: è uno stile architetturale in cui una risorsa (come un Cliente o un Ordine) viene esposta e risulta accessibile tramite le operazioni HTTP standard come GET o POST.
- **GraphQL**: è un protocollo che consente ai client (consumatori) di definire query specifiche per estrarre informazioni dai microservizi, filtrando i risultati per poter ottenere esattamente ciò che è stato richiesto.
- **Message broker**: si tratta di un middleware utilizzato per abilitare la comunicazione asincrona tra i servizi sfruttando le code.

gRPC

È un framework dedicato alla comunicazione remota sviluppato da Google, creato per far dialogare i microservizi tra loro in maniera molto efficiente. Le sue caratteristiche principali includono:

→ l'utilizzo del protocollo **HTTP/2**.

→ l'uso dei **Protocol Buffers** (protobuf) per la serializzazione dei dati, in sostituzione del più comune formato JSON.

→ è un framework **tipizzato**.

Microservizi con Spring Boot

Per sviluppare microservizi utilizzando Spring Boot all'interno di Visual Studio Code, è necessario seguire i seguenti passaggi:

- installare su VS Code l'estensione Spring Boot Extension Pack per impostare l'ambiente di lavoro.
- creare il progetto utilizzando Spring Initializr (integrato in VS Code o tramite il sito start.spring.io), oppure scaricando un progetto base già configurato dal sito di Spring.
- assicurarsi che il progetto utilizzi il modulo Spring Web, fondamentale per poter implementare e disporre di API REST.

Componenti per gRPC

Un'architettura basata su gRPC all'interno di Spring Boot si divide in tre componenti fondamentali:

1. **Contratto**: un file in formato protobuf che definisce formalmente il servizio e i suoi metodi (ad esempio, service HelloService con le relative richieste e risposte).

```
service HelloService {
     rpc SayHello (HelloRequest) returns (HelloResponse);
}
```

2. **Classe server**: una classe Java che viene eseguita sul server Spring Boot, annotata con @GrpcService, che estende la base generata dal contratto e implementa la logica reale del metodo (es, HelloServiceImpl).

```
@GrpcService
public class HelloServiceImpl extends HelloServiceGrpc.HelloServiceImpl
Base {

     @Override
     public void sayHello(HelloRequest request, StreamObserver<HelloRes
ponse> responseObserver) {
         HelloResponse response = HelloResponse.newBuilder().setMessage
("Ciao " + name).build();
         responseObserver.onNext(response);
     }
}
```

3. **Classe client**: un componente Spring Boot, annotato con @Service, che funge da client. Al suo interno utilizza l'annotazione @GrpcClient per richiamare lo stub (il riferimento locale del servizio remoto) ed effettuare fisicamente la chiamata passando i parametri richiesti.

```
@Service
public class HelloClient {
     @GrpcClient("hello-service")
     private HelloServiceGrpc.HelloServiceBlockingStub stub;

     public String call(String name) {
         HelloRequest request = HelloRequest.newBuilder().setName(nam
e).build();
         return stub.sayHello(request).getMessage();
     }
}
```

Sviluppo del microservizio

Lo sviluppo base di un microservizio consiste nella creazione di una API REST all'interno della classe principale dell'applicazione. Aprendo il file dell'applicazione (es. DemoApplication.java), si mappano le richieste HTTP ai metodi Java utilizzando le apposite annotazioni. Ad esempio, si può creare un metodo sayHello() e annotarlo con @GetMapping("/helloSpr") affinché restituisca una specifica stringa di testo quando quell'endpoint viene chiamato.

```
@SpringBootApplication
@RestController
```

```
public class DemoApplication {
     public static void main(String[] args) {
             SpringApplication.run(DemobApplication.class, args);
     }

     @GetMapping("/hello")
     public String hello(@RequestParam(value = "name", defaultValue = "Wor
ld") String name) {
             return String.format("Hello %s!", name);
     }

     @GetMapping("/helloSpr")
     public String sayHello() {
             return "Hello, Spring Boot!";
     }
}
```

Esecuzione

Una volta sviluppato, il microservizio può essere mandato in esecuzione direttamente dal terminale utilizzando il comando `mvn spring-boot:run`. A esecuzione avviata, è possibile interrogare l'applicazione tramite il browser, navigando verso gli URL esposti dalle API (ad esempio http://localhost:8080/hello o http://localhost:8080/helloSpr), avendo anche la possibilità di passare eventuali parametri nell'URL (es. http://localhost:8080/hello?name=Emi).

Inversion of Control

L'inversion of control (IoC) è un principio di progettazione adottato da Spring Boot in cui gli oggetti (o istanze) non vengono creati implementando codice diretto nell'applicazione, ma è il framework stesso a occuparsi della loro creazione. Questa tecnica è gestita dallo Sping IoC Container (o ApplicationContext), il quale crea gli oggetti, chiamati bean, e ne amministra sia il ciclo di vita che i reciproci collegamenti.

```
@Service
public class OrderService {
     private final OrderRepository orderRepository;

     public OrderService(OrderRepository orderRepository) {
             this.orderRepository = orderRepository;
     }
}

@Repository
public class OrderRepository {
}
```

Dependency Injection

La dependency injection (DI) è la tecnica pratica utilizzata per realizzare l'IoC: una classe riceve le proprie dipendenze dall'esterno invece di crearle al suo interno. Spring Boot utilizza annotazioni specifiche, come @Service e @Repository, per riconoscere le classi e iniettare automaticamente le istanze corrette laddove sono necessarie (ad esempio, iniettando un OrderRepository in un OrderService). La modalità raccomandata è la Constructor Injection (iniezione tramite costruttore):

```
public OrderService(OrderRepository orderRepository) { ...
```

che garantisce immutabilità, testabilità e rende le dipendenze immediatamente evidenti. Esistono inoltre alternative come la Field Injection (usando l'annotazione @Autowired direttamente su un campo):

```
@Autowired
private OrderRepository repo;
```

o la Setter Injection:

```
@Autowired
public void setRepo(OrderRepository repo) {
        this.repo = repo;
}
```

Vantaggi forniti da IoC e DI

L'utilizzo congiunto di IoC e DI porta diversi vantaggi architetturali:

- **Disaccoppiamento**: una classe che dipende da un'interfaccia può accettare in input un'istanza di qualunque classe implementi quell'interfaccia.
- **Testabilità**: testare il codice è più facile poiché si possono iniettare agevolmente oggetti mock o stub.
- **Flessibilità**: essendoci meno dipendenze rigide, il codice è molto più semplice da far evolvere nel tempo.
- **Automazione**: Spring ha il controllo completo sulla creazione dei bean e inietta le dipendenze in modo del tutto automatico.

IoC e dati persistenti

All'interno dell'applicazione, i dati presenti in una tabella del database sono modellati attraverso quattro componenti principali: Entity, Repository, Service e Controller. L'Entity rappresenta il dato stesso, che viene mappato sulla tabella del database tramite JPA (Java Persistence API):

```
@Entity
@Table(name = "users")
public class User {
     @Id
     @GeneratedValue(strategy = GenerationType.IDENTITY)
     private Long id;
     private String email;
```

```
     private String name;
}
```

Il Repository fornisce i metodi per accedere ai dati della tabella:

```
@Repository
public interface UserRepository extends JpaRepository<User, Long> {
        Optional<User> findByEmail(String email);
}
```

Grazie alla DI, Spring JPA crea automaticamente la classe concreta del repository, la registra come bean e la inietta dove richiesto.

Dependency Injection su Service

Il Service è il componente che gestisce la logica e utilizza il repository che gli viene iniettato da Spring. Spring si fa carico di generare il codice per il UserRepository (che recupererà i dati dal database tramite SQL o altre modalità) e inietta un'istanza di questo repository all'interno del costruttore della classe Service.

```
@Service
public class UserService {
        private final UserRepository userRepository;

        public UserService(UserRepository userRepository) {
                this.userRepository = userRepository;
        }

        public User getUserByEmail(String email) {
                return userRepository
                                        .findByEmail(email)
                                        .orElseThrow(() -> new RuntimeExceptio
n("User not found"));
        }
}
```

Controller

Il Controller ha il compito di ricevere le chiamate provenienti dal client e di utilizzare il Service per elaborarle. Questo componente viene configurato tramite le annotazioni @RestController e @RequestMapping, che permettono di definire il percorso (path) dell'endpoint dell'URL e di associarlo a uno specifico metodo Java (usando annotazioni come @GetMapping o @PostMapping). Anche in questo livello, Spring inietta automaticamente l'istanza del Service nel controller, occupandosi di collegare tutti gli oggetti necessari al flusso di esecuzione.

```
@RestController
@RequestMapping("/users")
public class UserController {
         private final UserService userService;
```

```
        public UserController(UserService userService) {
                this.userService = userService;
        }

        @GetMapping("/{email}")
        public User getUser(@PathVariable String email) {
                return userService.getUserByEmail(email);
        }
}
```

Annotazioni di Spring Boot

Per sviluppare microservizi con Spring Boot si utilizzano diverse annotazioni specifiche:

- **@SpringBootApplication**: indica che la classe è il punto di avvio dell'applicazione (contiene il metodo main).
- **@Service**: indica una classe che contiene la business logic.
- **@RestController**: indica una classe che funge da controller e che espone API REST.
- **@RequestMapping**: serve ad associare i metodi del controller alle specifiche richieste HTTP e a definire i percorsi (path).
- **@GetMapping**, **@PostMapping**, **@PutMapping**, **@DeleteMapping**: sono le annotazioni utilizzate per mappare le corrispondenti operazioni REST.
- **@RequestParam**: permette di estrarre e prendere i parametri passati tramite la richiesta (ad esempio dall'URL).

Ulteriori annotazioni fondamentali sono:

- per la **dependency injection**: l'annotazione **@Autowired**, che viene utilizzata per iniettare le dipendenze in modo automatico.
- per la **persistenza dei dati**: l'annotazione **@Entity**, che serve a indicare che una classe è un'entità JPA, la quale verrà mappata su una tabella all'interno del database.

Servizi REST

Un servizio REST (*Representational State Transfer*) è un servizio web progettato per facilitare la comunicazione tra componenti su Internet. Le caratteristiche principali che definiscono un servizio REST sono:

- **Statelessness** (comunicazione stateless): il server non conserva informazioni tra una richiesta e la successiva. La comunicazione avviene tramite un modello richiesta-risposta in cui ogni singola richiesta deve contenere tutte le informazioni necessarie al server per ricostruire lo stato ed elaborare la chiamata.
- **Risorse**: tutti i dati sono rappresentati come risorse, e ciascuna risorsa è identificata in modo univoco da un URI (Uniform Resource Identifier).
- **Interfaccia uniforme e operazioni CRUD**: i servizi REST si basano su un'interfaccia uniforme e consistente che sfrutta i metodi standard HTTP per interagire con le risorse. Nello specifico, supportano le operazioni base (Create, Read, Update, Delete) che sono mappate come segue:

    - **POST** per creare.
    - **GET** per leggere.
    - **PUT** per aggiornare.
    - **DELETE** per rimuovere.
- **Rappresentazione**: le risorse vengono scambiate in un formato specifico (molto spesso JSON o XML). È il client a specificare quale rappresentazione desidera tramite l'header della richiesta.
- **Messaggi auto-descrittivi**: le risposte fornite dal server devono contenere tutte le informazioni per essere interpretate, sfruttando a questo scopo gli header HTTP e i codici di stato.
- **Hypermedia links**: si possono utilizzare collegamenti ipertestuali per permettere ai client di scoprire ulteriori risorse disponibili.
- **Sistemi a strati**: i servizi REST dovrebbero essere strutturati in modo da consentire l'aggiunta di componenti intermediari, come ad esempio load balancer, cache o sistemi di sicurezza.